Processingをはじめよう

第6章

移動、回転、伸縮

目次

- 座標移動の基礎
- 回転
- 伸縮
- PushとPop
- Robot4: Translate, Rotate, Scale

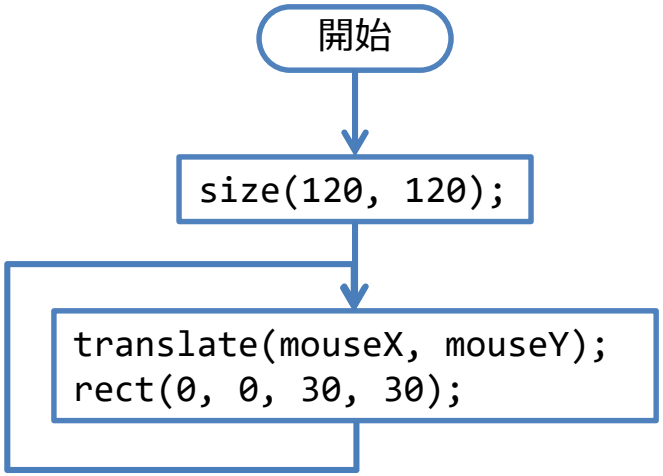
座標移動の基礎

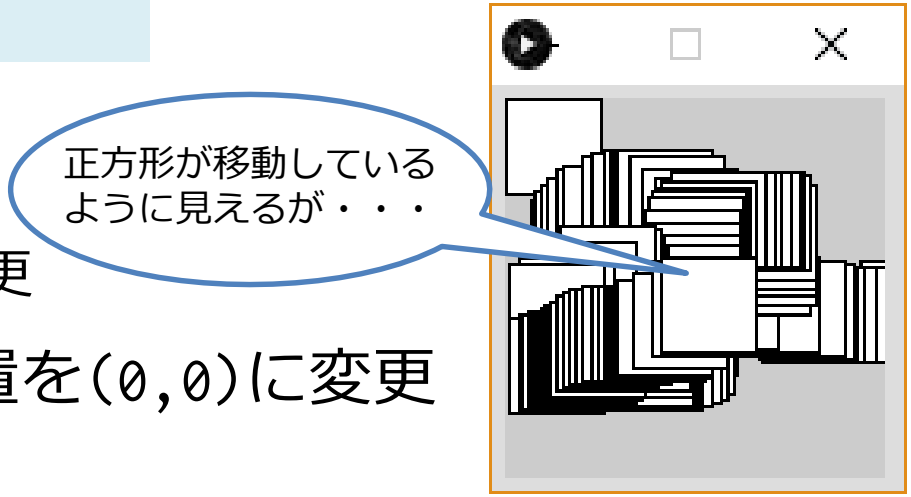
- 座標を変更して、移動、回転、伸縮する
  - 図形は変更せず、座標の方を変える
- 座標変換 教科書 P242
  - 位置をずらす `translate(x, y)`
  - 回転させる `rotate(a)`
  - 拡大・縮小する `scale(x, y)`
  - 座標を保存する `pushMatrix()`
  - 座標を復元する `popMatrix()`

座標移動の基礎

- Example 6-1 位置の変更

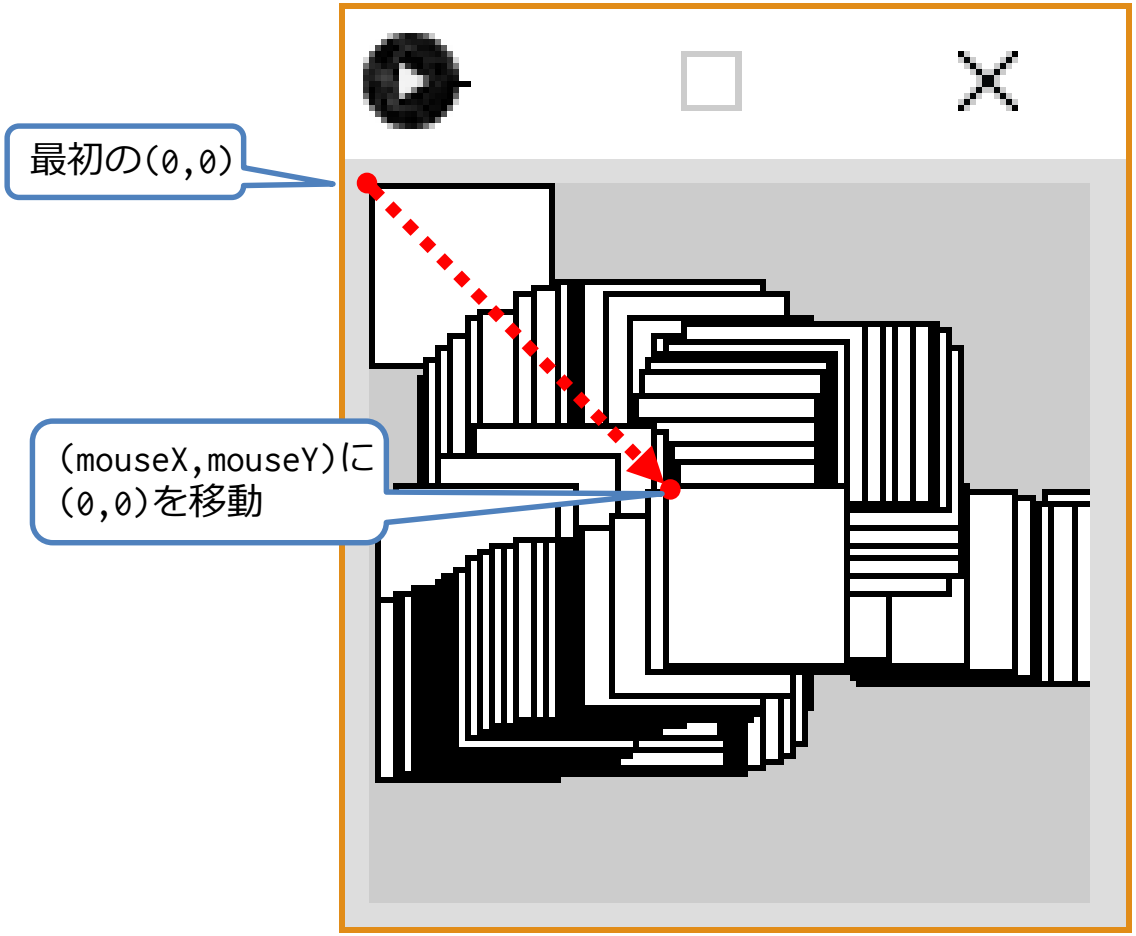
```
void setup() {
  size(120, 120);
}

void draw() {
  translate(mouseX, mouseY);
  rect(0, 0, 30, 30);
}
```





- translate(x, y)
  - (x,y)を原点(0, 0)に変更
- マウスカーソルの位置を(0,0)に変更

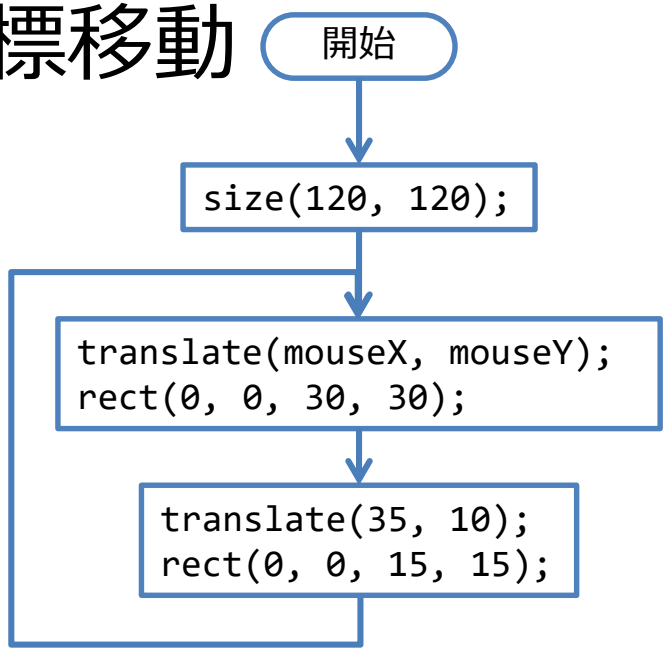
実行結果



座標移動の基礎

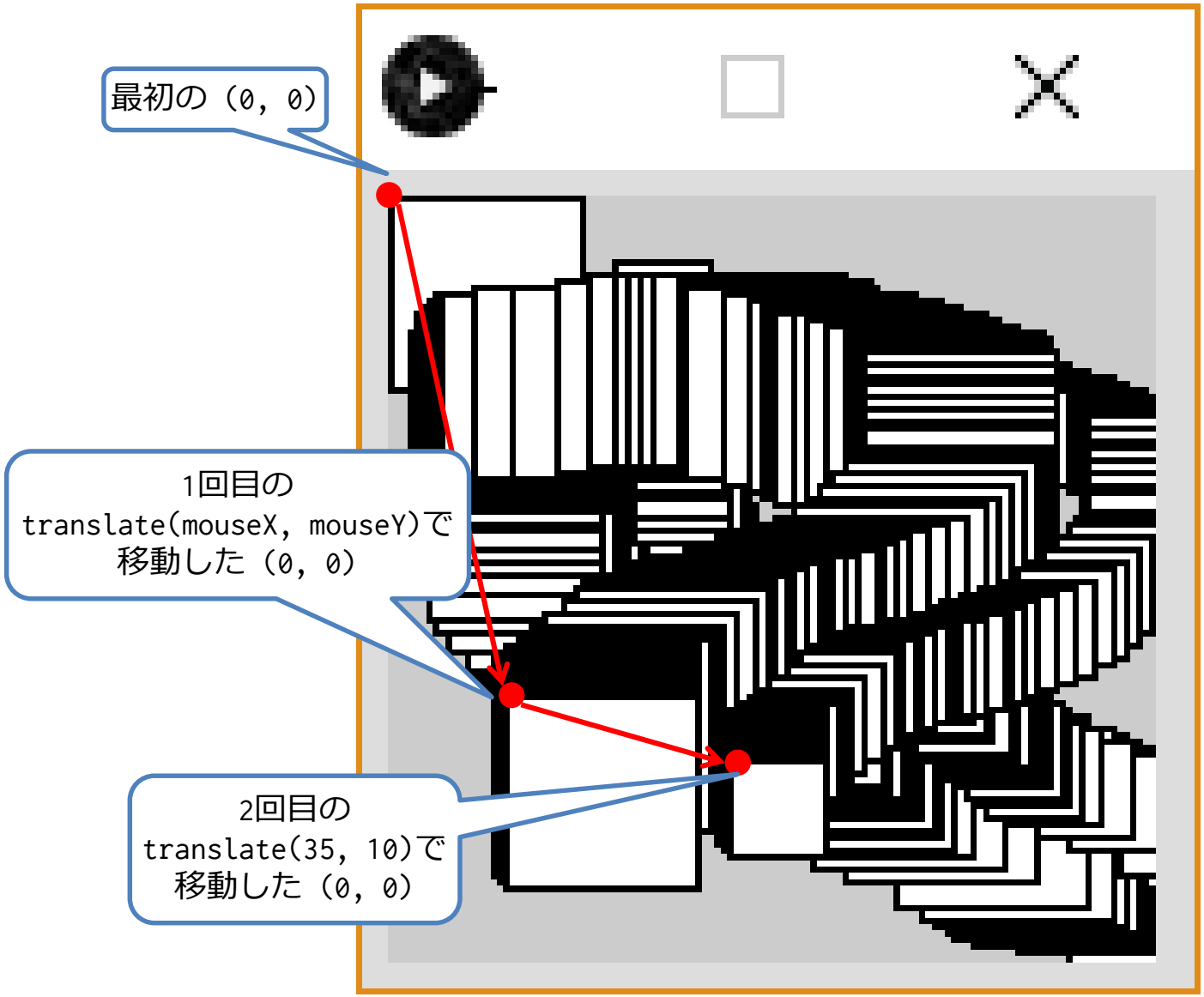
- Example 6-2 複数回の座標移動

```
void setup() {
  size(120, 120);
}

void draw() {
  translate(mouseX, mouseY);
  rect(0, 0, 30, 30);
  translate(35, 10);
  rect(0, 0, 15, 15);
}
```



- 座標移動して1つ目の正方形を描き
  さらに座標移動して2つ目の正方形を描く

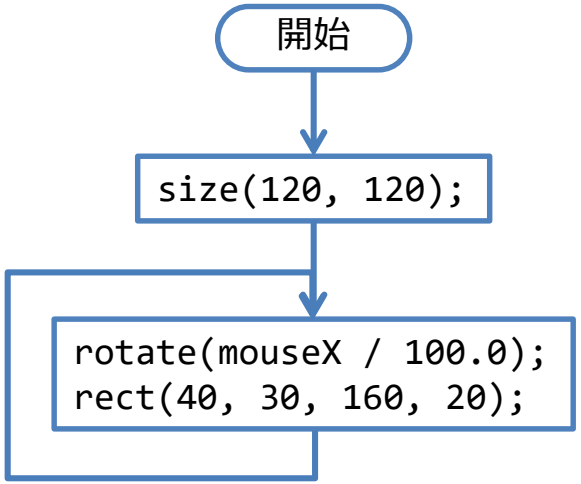
Example 6-2


最初の（0，0）
1回目の
translate(mouseX，mouseY)で
移動した（0，0）
2回目の
translate(35，10)で
移動した（0，0）

回転

- Example 6-3 図形の角を軸に回転

```
void setup() {
  size(120, 120);
}

void draw() {
  rotate(mouseX / 100.0);
  rect(40, 30, 160, 20);
}
```

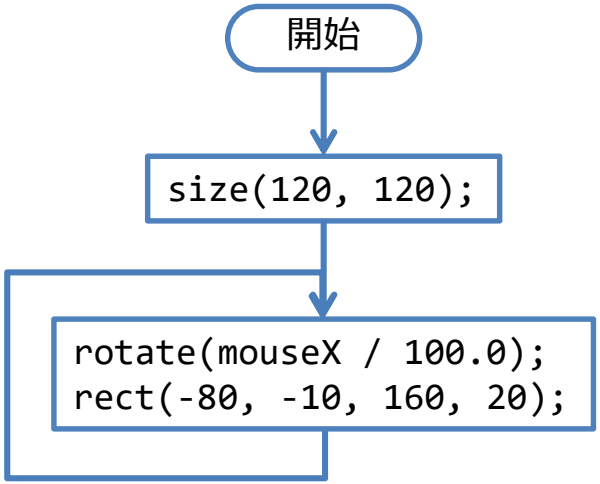


- rotate(角度)
  - 角度はラジアン単位

回転

- Example 6-4 図形の中心を軸に回転

```
void setup() {
  size(120, 120);
}

void draw() {
  rotate(mouseX / 100.0);
  rect(-80, -10, 160, 20);
}
```

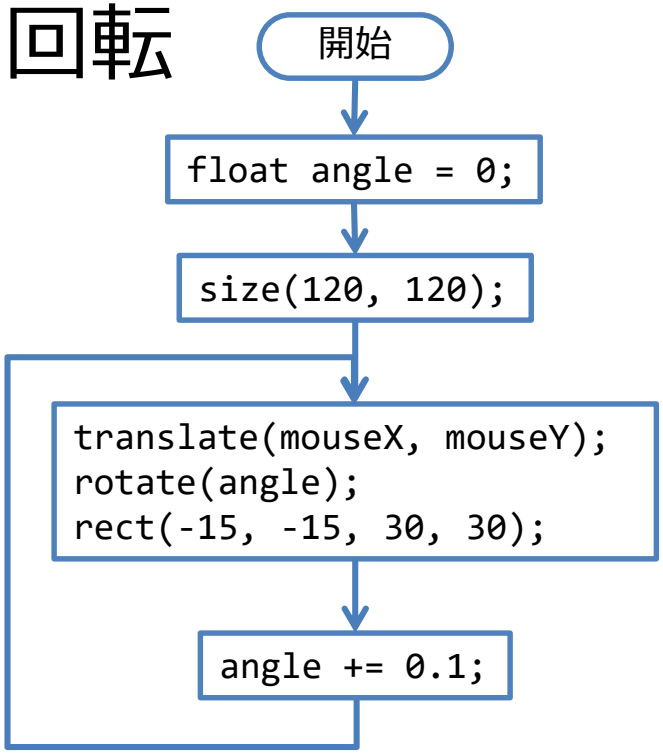


- 長方形の中心を軸にして回転
  - 実行ウィンドウから
    はみ出て見えていないが

回転

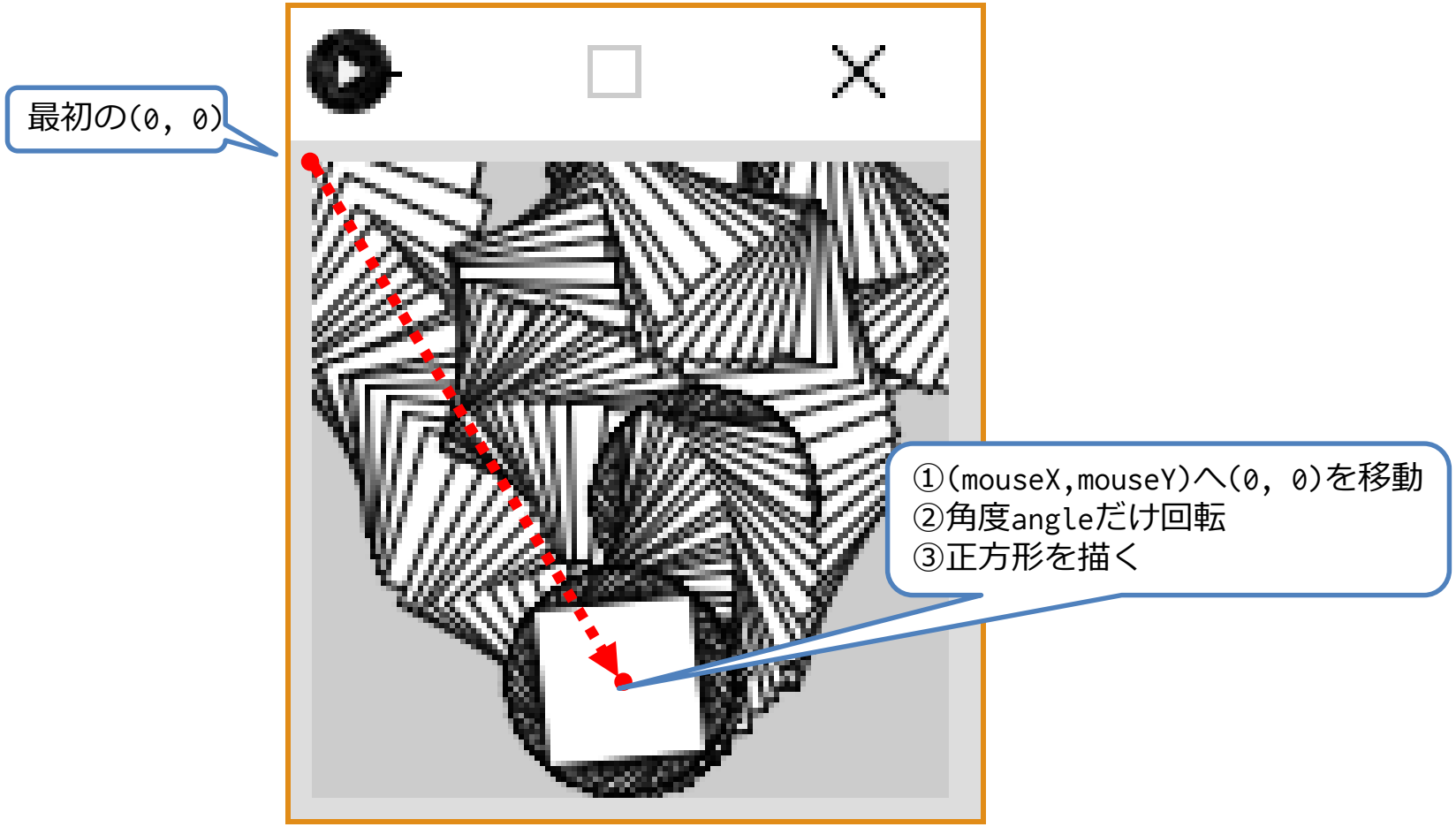
- Example 6-5 移動してから回転

```
float angle = 0;

void setup() {
  size(120, 120);
}

void draw() {
  translate(mouseX, mouseY);
  rotate(angle);
  rect(-15, -15, 30, 30);
  angle += 0.1;
}
```



- マウスカーソルを(0, 0)、
  そこで図形の中心を軸に回転

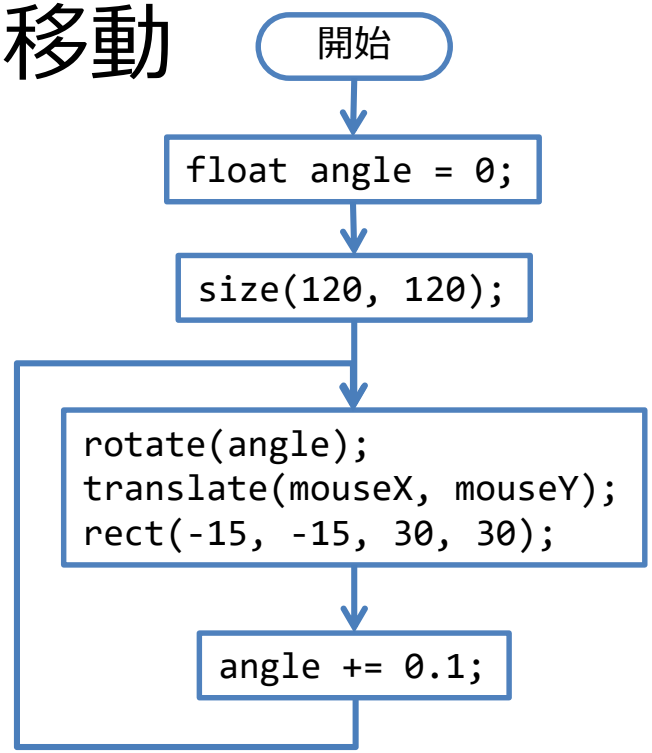
実行結果

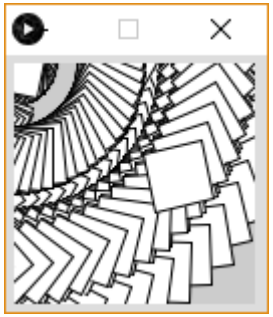


回転

- Example 6-6 回転してから移動

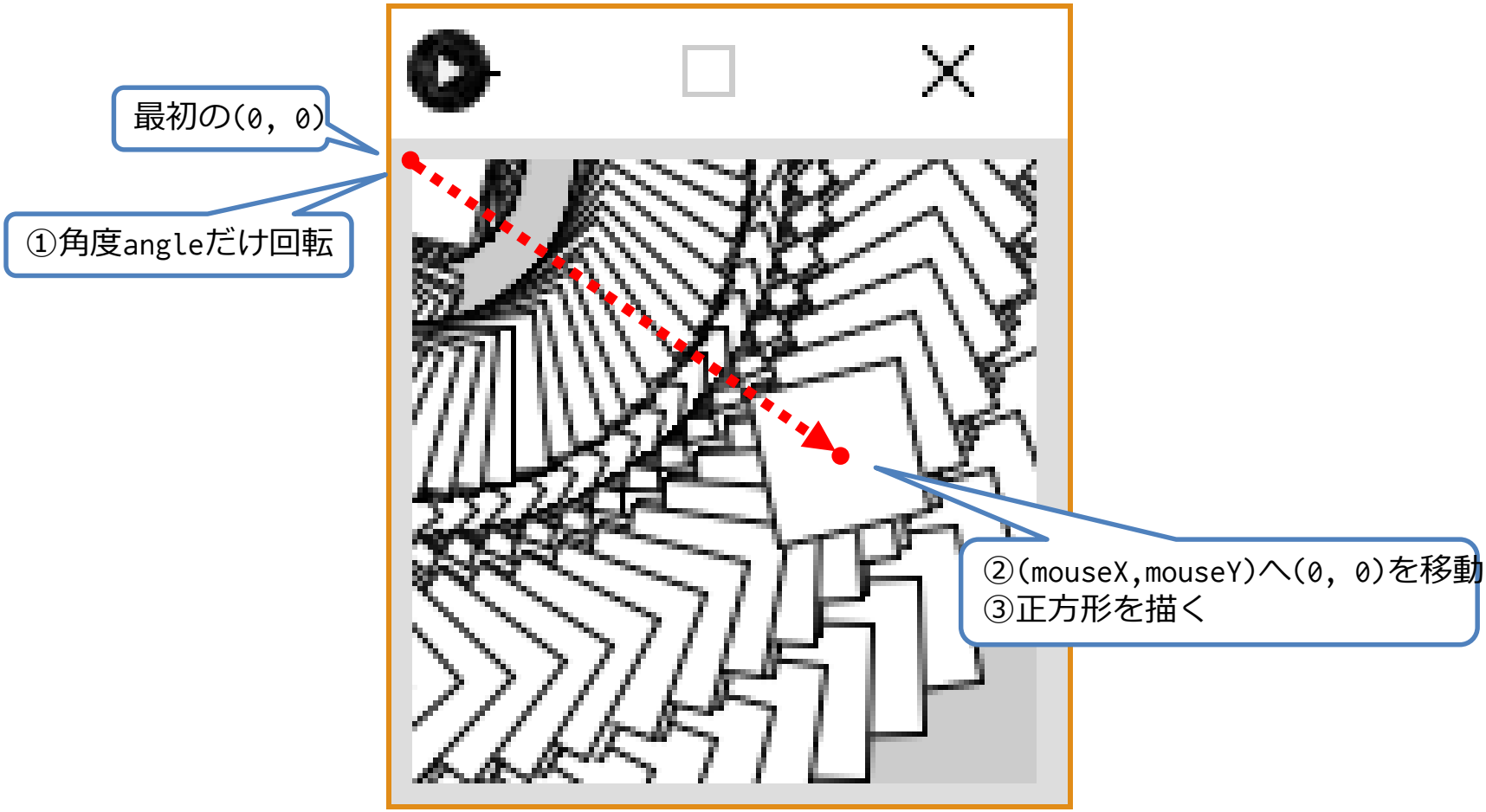
```
float angle = 0.0;

void setup() {
  size(120, 120);
}

void draw() {
  rotate(angle);
  translate(mouseX, mouseY);
  rect(-15, -15, 30, 30);
  angle += 0.1;
}
```





- (0, 0)で回転して
  (mouseX, mouseY)に(0, 0)を移動

実行結果



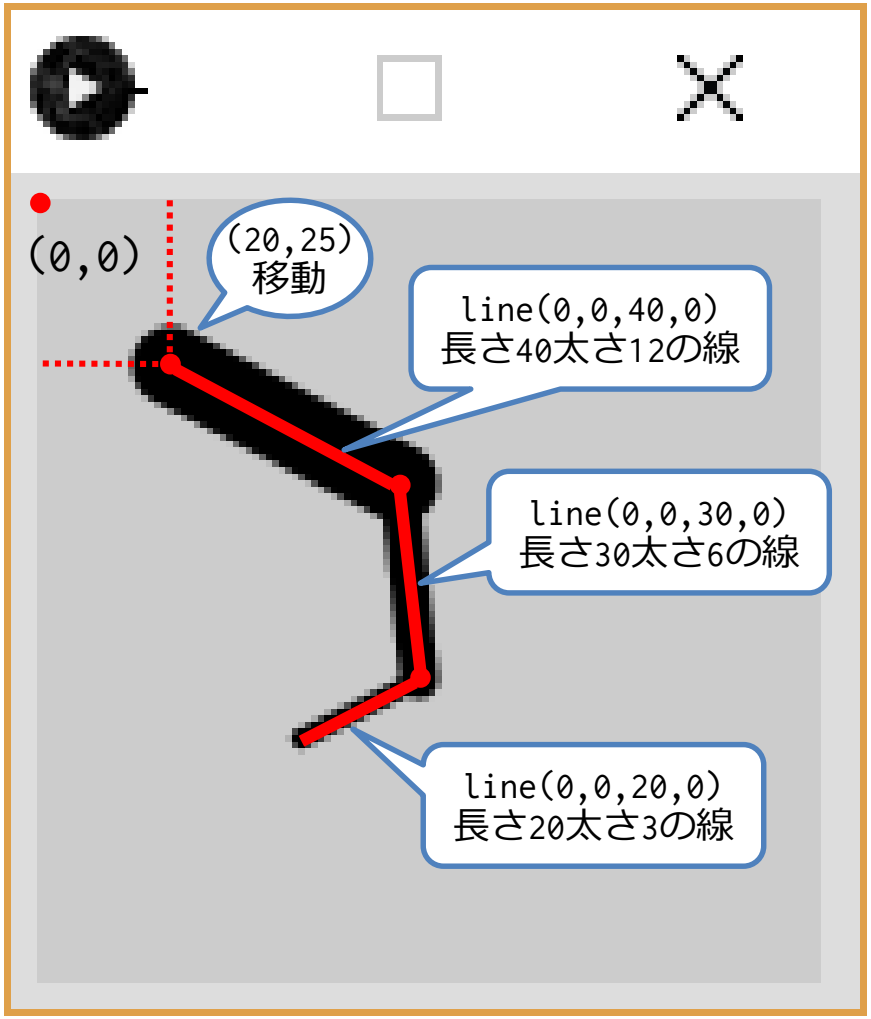
回転

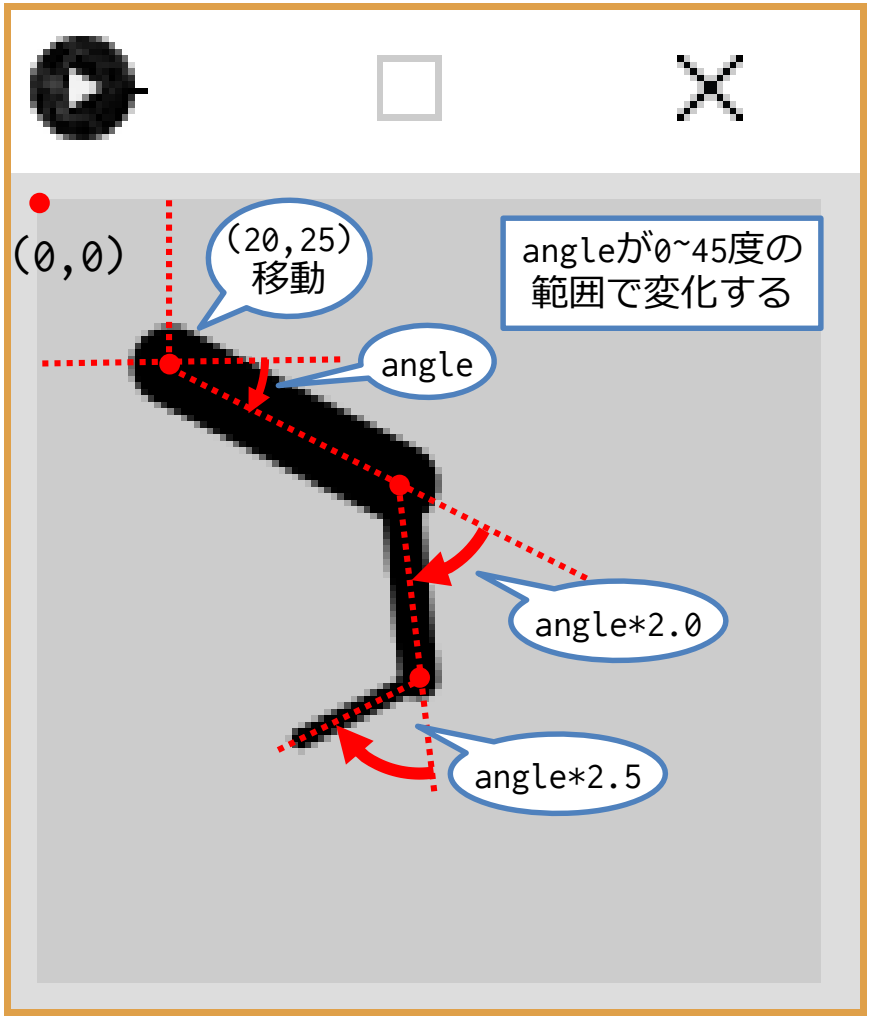
- Example 6-7 関節のある腕

```
float angle = 0.0;
float angleDirection = 1;
float speed = 0.005;

void setup() {
  size(120, 120);
}
void draw() {
  background(204);
  //腕1
  translate(20, 25);
  rotate(angle);
  strokeWeight(12);
  line(0, 0, 40, 0);
```

```
  //腕2
  translate(40, 0);
  rotate(angle * 2.0);
  strokeWeight(6);
  line(0, 0, 30, 0);
  //腕3
  translate(30, 0);
  rotate(angle * 2.5);
  strokeWeight(3);
  line(0, 0, 20, 0);
  //角度の更新
  angle += speed * angleDirection;
  if ((angle > QUARTER_PI) || (angle < 0)) {
    angleDirection = -angleDirection;
  }
}
```

実行結果

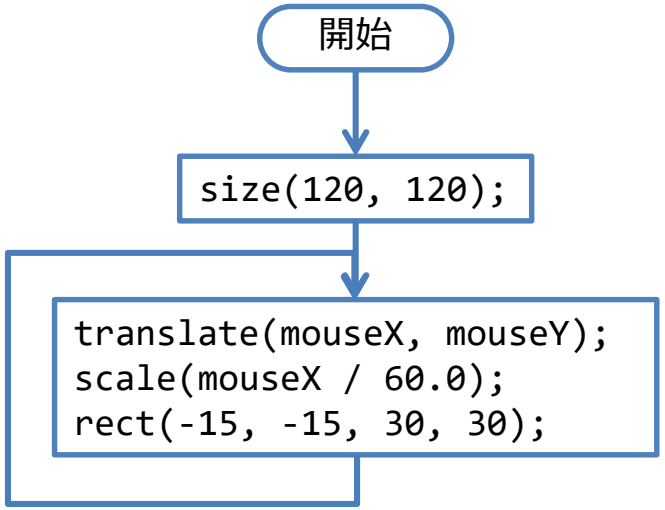

(0,0)
(20,25)
移動
line(0,0,40,0)
長さ40太さ12の線
line(0,0,30,0)
長さ30太さ6の線
line(0,0,20,0)
長さ20太さ3の線


(0,0)
(20,25)
移動
angleが0~45度の
範囲で変化する
angle
angle*2.0
angle*2.5

伸縮

- Example 6-8 伸縮

```
void setup() {
  size(120, 120);
}

void draw() {
  translate(mouseX, mouseY);
  scale(mouseX / 60.0);
  rect(-15, -15, 30, 30);
}
```

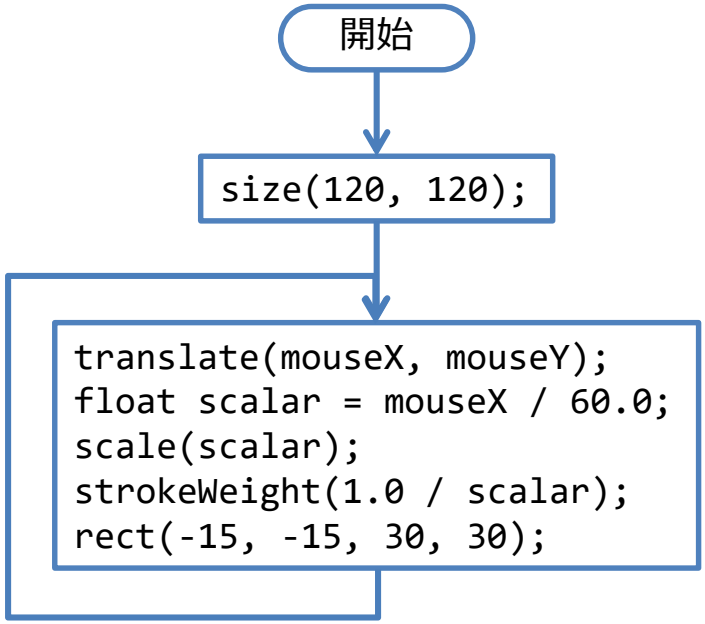


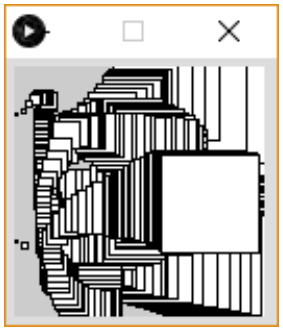
- scale(倍率)
  - 座標を指定した倍率で伸縮する

伸縮

- Example 6-9 線の太さを一定に保つ

```
void setup() {
  size(120, 120);
}

void draw() {
  translate(mouseX, mouseY);
  float scalar = mouseX / 60.0;
  scale(scalar);
  strokeWeight(1.0 / scalar);
  rect(-15, -15, 30, 30);
}
```



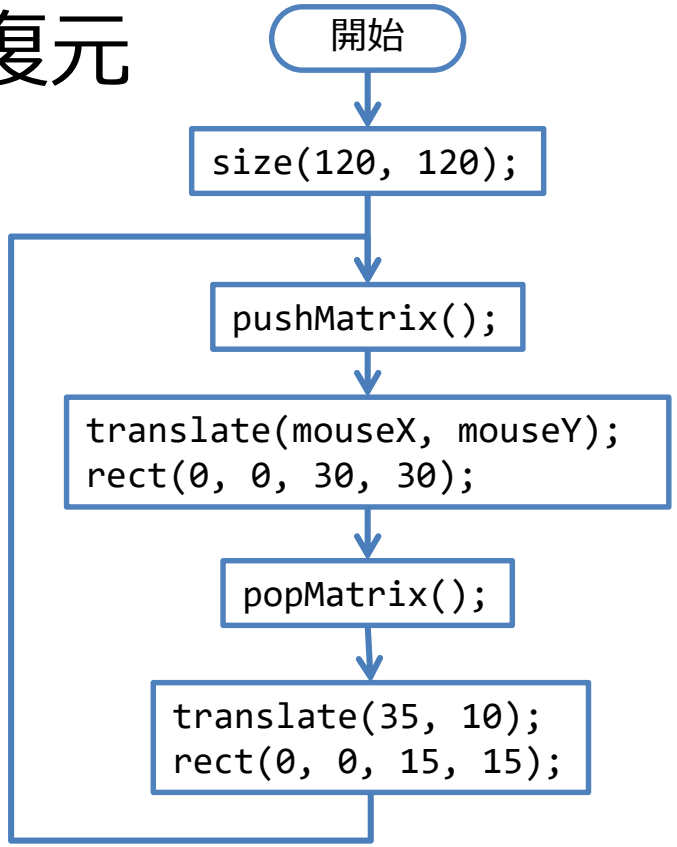


- 伸縮時に線の太さを(1/倍率)に設定することで一定に保つ

PushとPop

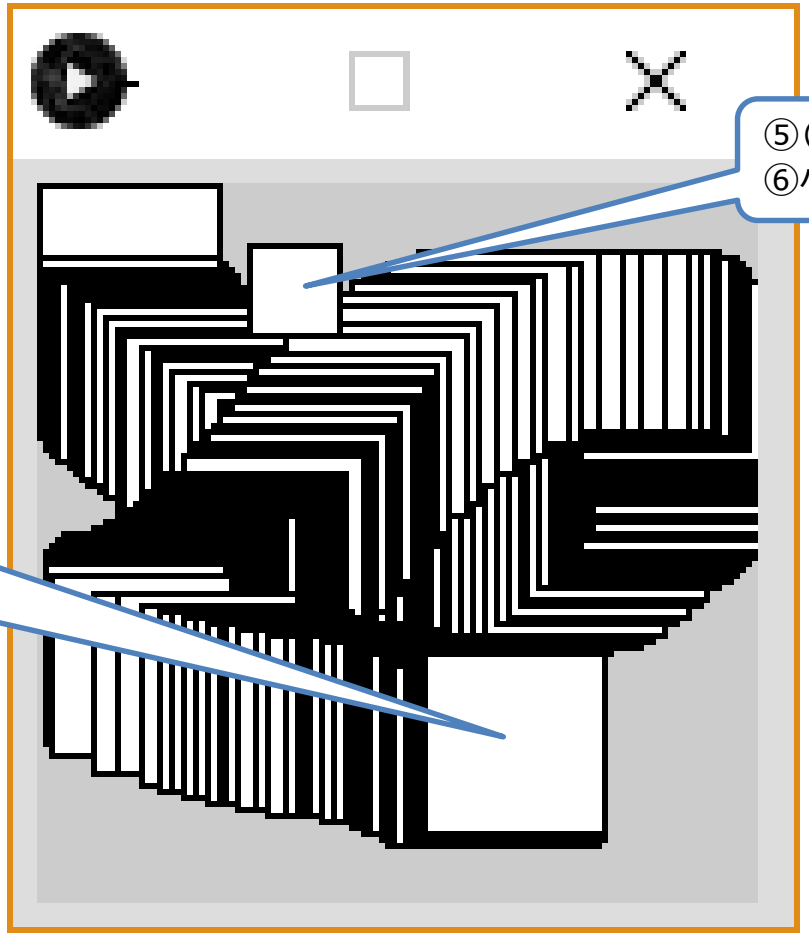
- Example 6-10 座標系の復元

```
void setup() {
  size(120, 120);
}

void draw() {
  pushMatrix();
  translate(mouseX, mouseY);
  rect(0, 0, 30, 30);
  popMatrix();
  translate(35, 10);
  rect(0, 0, 15, 15);
}
```



- pushMatrix()とpopMatrix()は必ずペアで使う
  - pushMatrix()　　この時点の(0, 0)の位置を保存
  - popMatrix()　　保存した(0, 0)の位置を復元

実行結果

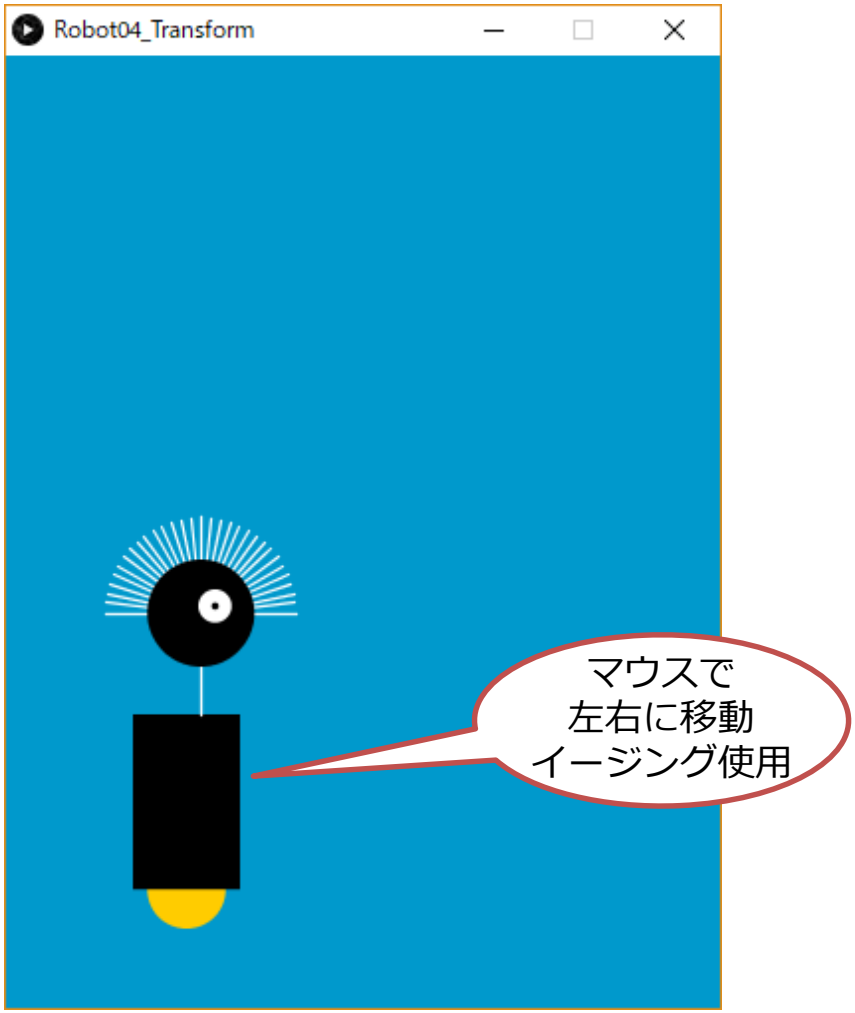
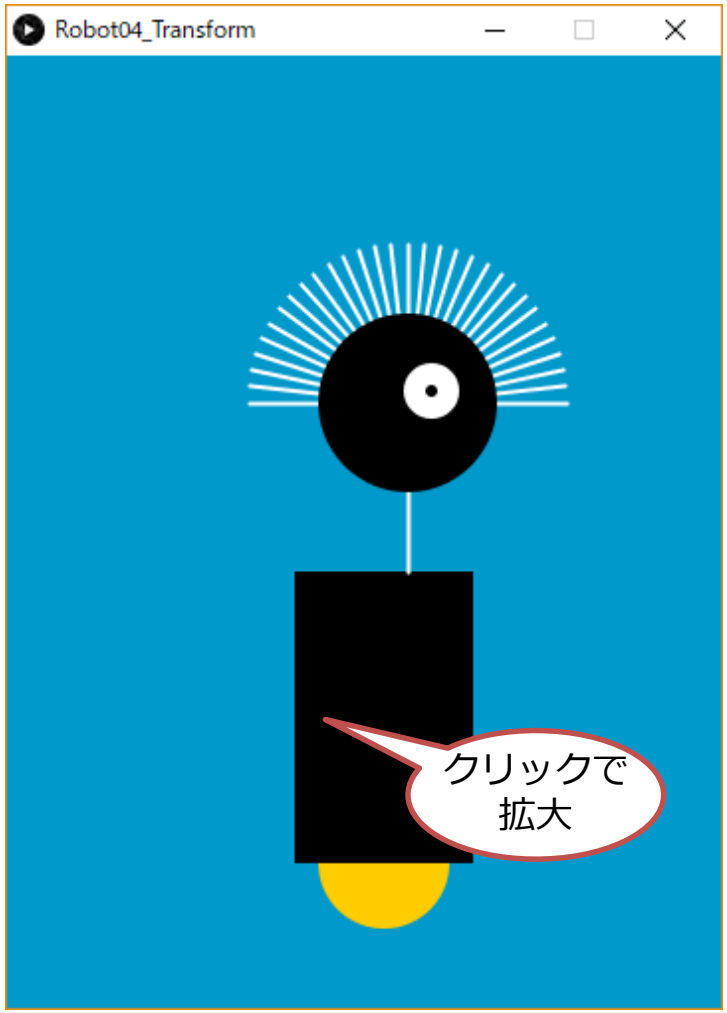


⑤(35,10)移動
⑥小さい正方形を描く

①現在の座標系を保存
②(mouseX,mouseY)に(0,0)移動
③正方形を描く
④元の座標に戻す

Robot 4: Translate, Rotate, Scale

```
float x = 60;
float y = 440;
int radius = 45;
int bodyHeight = 180;
int neckHeight = 40;

float easing = 0.04;

void setup() {
  size(360, 480);
  ellipseMode(RADIUS);
}
void draw() {
  strokeWeight(2);
  float neckY = -1 * (bodyHeight +
                 neckHeight + radius);
  background(0, 153, 204);
  translate(mouseX, y);
  if (mousePressed) {
    scale(1.0);
  } else {
    scale(0.6);
  }
 noStroke();
 fill(255, 204, 0);
```

```
  ellipse(0, -33, 33, 33);
  fill(0);
  rect(-45, -bodyHeight, 90, bodyHeight-33);

  stroke(255);
  line(12, -bodyHeight, 12, neckY);

  pushMatrix();
  translate(12, neckY);
  float angle = -PI/30.0;
  for (int i = 0; i <= 30; i++) {
    line(80, 0, 0, 0);
    rotate(angle);
  }
  popMatrix();

  noStroke();
  fill(0);
  ellipse(12, neckY, radius, radius);
  fill(255);
  ellipse(24, neckY-6, 14, 14);
  fill(0);
  ellipse(24, neckY-6, 3, 3);
}
```

Robot 4: Translate, Rotate, Scale